

PHP - Decision Making

A computer program by default follows a simple input-process-output path sequentially. This sequential flow can be altered with the decision control statements offered by all the computer programming languages including PHP.

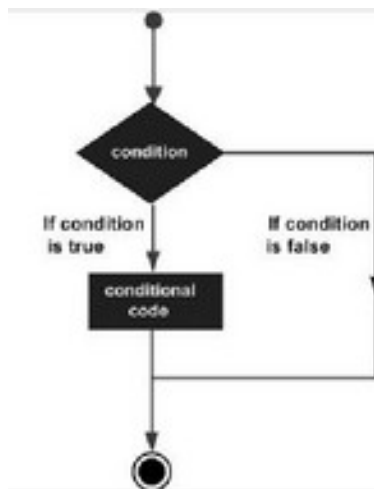
Decision Making in a Computer Program

Decision-making is the anticipation of conditions occurring during the execution of a program and specified actions taken according to the conditions.

You can use conditional statements in your code to make your decisions. The ability to implement conditional logic is one of the fundamental requirements of a programming language.

A Typical Decision Making Structure

Following is the general form of a typical decision making structure found in most of the programming languages



Decision Making Statements in PHP

PHP supports the following three decision making statements

- **if...else statement:** Use this statement if you want to execute a set of code when a condition is true and another if the condition is not true.
- **elseif statement:** Use this statement with the if...else statement to execute a set of code if one of the several conditions is true

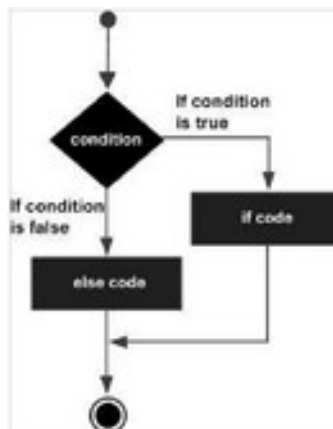
- **switch statement:** If you want to select one of many blocks of code to be executed, use the Switch statement. The **switch** statement is used to avoid long blocks of **if..elseif..else** code.

Almost all the programming languages (including PHP) define the if-else statements. It allows for conditional execution of code fragments. The syntax for using the if-else statement in PHP is similar to that of C

```
if (expr)
    statement1
else
    statement2
```

The expression here is a Boolean expression, evaluating to true or false

- Any expression involving Boolean operators such as <, >, <=, >=, !=, etc. is a Boolean expression.
- If the expression results in true, the subsequent statement it may be a simple or a compound statement i.e., a group of statements included in pair of braces will be executed.
- If the expression is false, the subsequent statement is ignored, and the program flow continues from next statement onwards.
- The use of **else** statement is optional. If the program logic requires another statement or a set of statements to be executed in case the expression (after the **if** keyword) evaluates to false.



The **elseif** statement is a combination of **if** and **else**. It allows you to check multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE. Just like the **else** statement, the **elseif** statement is optional.

The **switch** statement is similar to a series of **if** statements on the same expression. We shall learn about these statements in detail in the subsequent chapters of this tutorial.

Nested if Statement

You can use an if statement within another if statement to verify multiple conditions in a logical way.

```
<?php
    $age = 20;
    if ($age >= 18) {
        if ($age < 60) {
            echo "You are an adult.";
        } else {
            echo "You are a senior citizen.";
        }
    } else {
        echo "You are a minor.";
    }
?>
```

Output

Here is the outcome of the following code

You are an adult.

Break and Continue in Decision Making

Here we will show you the usage of break and continue keywords in decision making. So break is used to stop execution in a loop. And continue is used to skip the current iteration and moves to the next iteration in a loop.

```
<?php
    $day = "Monday";
    switch ($day) {
        case "Monday":
            echo "Start of the week!";
            break;
        case "Friday":
            echo "Weekend is near!";
            break;
        default:
            echo "Have a nice day!";
    }
?>
```

Output

Here is the outcome of the following code

Start of the week!

PHP - IfElse Statement

The ability to implement conditional logic is the fundamental requirement of any programming language (PHP included). PHP has three keywords (also called as **language constructs**) - **if**, **elseif** and **else** - are used to take decision based on the different conditions.

The **if** keyword is the basic construct for the conditional execution of code fragments. More often than not, the **if** keyword is used in conjunction with **else** keyword, although it is not always mandatory.

If you want to execute some code if a condition is true and another code if the same condition is false, then use the "if....else" statement.

Syntax

The usage and syntax of the **if** statement in PHP is similar to that of the C language. Here is the syntax of **if** statement in PHP

```
if (expression)
    code to be executed if expression is true;
else
    code to be executed if expression is false;
```

The **if** statement is always followed by a Boolean expression.

- PHP will execute the statement following the Boolean expression if it evaluates to true.
- If the Boolean expression evaluates to false, the statement is ignored.
- If the algorithm needs to execute another statement when the expression is false, it is written after the **else** keyword.

Example

Here is a simple PHP code that demonstrates the usage of **if else** statements. There are two variables \$a and \$b. The code identifies which one of them is bigger.

```
<?php
    $a=10;
    $b=20;
    if ($a > $b)
        echo "a is bigger than b";
    else
        echo "a is not bigger than b";
?>
```

When the above code is run, it displays the following **output**

a is not bigger than b

Interchange the values of "a" and "b" and run again. Now, you will get the following output

a is bigger than b

Example

The following example will output "Have a nice weekend!" if the current day is Friday, else it will output "Have a nice day!"

```
<?php
    $d = date("D");

    if ($d == "Fri")
        echo "Have a nice weekend!";
    else
        echo "Have a nice day!";
?>
```

It will produce the following **output**

Have a nice weekend!

Using endif in PHP

PHP code is usually intermixed with HTML script. We can insert HTML code in the **if** part as well as the **else** part in PHP code. PHP offers an alternative syntax for **if** and **else** statements. Change the opening brace to a colon (:) and the closing brace to **endif**; so that a HTML block can be added to the **if** and **else** part.

```
<?php
    $d = date("D");

    if ($d == "Fri"): ?>
        <h2>Have a nice weekend!</h2>

    <?php else: ?>
        <h2>Have a nice day!</h2>
    <?php endif ?>
```

Make sure that the above script is in the document root of PHP server. Visit the URL **<http://localhost/hello.php>**. Following output should be displayed in the browser, if the current day is not a Friday

Have a nice day!

Using elseif in PHP

If you want to execute some code if one of the several conditions are true, then use the **elseif** statement. The **elseif** language construct in PHP is a combination of **if** and **else**.

- Similar to **else**, it specifies an alternative statement to be executed in case the original **if** expression evaluates to false.
- However, unlike **else**, it will execute that alternative expression only if the **elseif** conditional expression evaluates to true.

```
if (expr1)
    code to be executed if expr1 is true;
elseif (expr2)
    code to be executed if expr2 is true;
else
    code to be executed if expr2 is false;
```

Example

Let us modify the above code to display a different message on Sunday, Friday and other days.

```
<?php
    $d = date("D");
    if ($d == "Fri")
        echo "<h3>Have a nice weekend!</h3>";

    elseif ($d == "Sun")
        echo "<h3>Have a nice Sunday!</h3>";

    else
        echo "<h3>Have a nice day!</h3>";
?>
```

On a Sunday, the browser shall display the following **output**

Have a nice Sunday!

Example

Here is another example to show the use of **if-elseif-else** statements

```
<?php
    $x=13;
    if ($x%2==0) {
        if ($x%3==0)
            echo "<h3>$x is divisible by 2 and 3</h3>";
        else
            echo "<h3>$x is divisible by 2 but not divisible by 3</h3>";
    }

    elseif ($x%3==0)
        echo "<h3>$x is divisible by 3 but not divisible by 2</h3>";
```

```
    else
        echo "<h3>$x is not divisible by 3 and not divisible by 2</h3>";
?>
```

The above code also uses **nestedif** statements.

For the values of **x** as 13, 12 and 10, the **output** will be as follows

```
13 is not divisible by 3 and not divisible by 2
12 is divisible by 2 and 3
10 is divisible by 2 but not divisible by 3
```

PHP - Switch Statement

In PHP, the **switch** statement allows many **if-else** conditions for a single variable. Sometimes you need to compare a variable to multiple values and run separate code for each one. In general, you would write if elseifelse.

An excessive number of if-else statements can result in long, difficult-to-read code. Using a switch statement instead of many if-else statements can reduce code complexity and improve readability.

Using ifelseifelse Statements

The following PHP script uses if elseif statements

```
if ($x == 0) {  
    echo "x equals 0";  
} elseif ($x == 1) {  
    echo "i equals 1";  
} elseif ($x == 2) {  
    echo "x equals 2";  
}
```

Using switch Statement

You can get the same result by using the switch case statements as shown below

```
switch ($x) {  
    case 0:  
        echo "x equals 0";  
        break;  
    case 1:  
        echo "x equals 1";  
        break;  
    case 2:  
        echo "x equals 2";  
        break;  
}
```

The **switch** statement is followed by an expression, which is successively compared with value in each case clause. If it is found that the expression matches with any of the cases, the corresponding block of statements is executed.

- The switch statement executes the statements inside the curly brackets line by line.
- If and when a case statement is found whose expression evaluates to a value that matches the value of the switch expression, PHP starts to execute the statements until the end of the switch block, or the first time it encounters a **break** statement.

- If you don't write a **break** statement at the end of a case's statement list, PHP will go on executing the statements of the following case.

Without Break Statements

Try to run the above code by removing the breaks. If the value of x is 0, you will find that the output includes "x equals 1" as well as "x equals 2" lines.

```
<?php
    $x=0;
    switch ($x) {
        case 0:
            echo "x equals 0 \n";
        case 1:
            echo "x equals 1 \n";
        case 2:
            echo "x equals 2";
    }
?>
```

Output

It will produce the following output

```
x equals 0
x equals 1
x equals 2
```

Thus, it is important make sure to end each case block with a break statement.

Using default Case in Switch

A special case is the default case. This case matches anything that wasn't matched by the other cases. Using **default** is optional, but if used, it must be the last case inside the curly brackets.

You can club more than one cases to simulate multiple logical expressions combined with the **or** operator.

```
<?php
    $x=10;
    switch ($x) {
        case 0:
        case 1:
        case 2:
            echo "x between 0 and 2 \n";
            break;
        default:
            echo "x is less than 0 or greater than 2";
    }
?>
```

The values to be compared against are given in the **case clause**. The value can be a number, a string, or even a function. However you cannot use comparison operators (<, > == or !=) as a value in the **case clause**.

You can choose to use semicolon instead of colon in the case clause. If no matching case found, and there is no default branch either, then no code will be executed, just as if no **if** statement was true.

Using switch-endswitch Statement

PHP allows the usage of alternative syntax by delimiting the switch construct with **switch-endswitch** statements. The following version of switch case is acceptable.

```
<?php
    $x=0;
    switch ($x) :
        case 0:
            echo "x equals 0";
            break;
        case 1:
            echo "x equals 1 \n";
            break;
        case 2:
            echo "x equals 2 \n";
            break;
        default:
            echo "None of the above";
    endswitch
?>
```

Using switch to Display the Current Day

Obviously, you needn't write a **break** to terminate the default case, it being the last case in the switch construct.

Example

Take a look at the following example

```
<?php
    $d = date("D");

    switch ($d){
        case "Mon":
            echo "Today is Monday";
            break;

        case "Tue":
            echo "Today is Tuesday";
            break;

        case "Wed":
            echo "Today is Wednesday";
            break;
```

```

        case "Thu":
            echo "Today is Thursday";
            break;

        case "Fri":
            echo "Today is Friday";
            break;

        case "Sat":
            echo "Today is Saturday";
            break;

        case "Sun":
            echo "Today is Sunday";
            break;

        default:
            echo "Wonder which day is this ?";
    }
?>

```

Output

It will generate the following output

Today is Monday

Switch with Strings

Now we will use a switch statement with string values to find the user roles. So here is the program showing the usage of strings in a switch case

```

<?php
    $role = "admin";

    switch ($role) {
        case "admin":
            echo "Welcome, Admin! You have full access.";
            break;

        case "editor":
            echo "Hello, Editor! You can edit content.";
            break;

        case "subscriber":
            echo "Hi, Subscriber! You can read articles.";
            break;

        default:
            echo "Unknown role. Please contact support.";
    }
?>

```

Output

Here is the outcome of the following code

Welcome, Admin! You have full access.

Switch with Arithmetic Operations

In the below PHP code we will try to perform calculations as per the user input. Check the below program for the demonstration of switch case with arithmetic operations

```
<?php
$operation = "+";
$a = 10;
$b = 5;

switch ($operation) {
    case "+":
        echo "Addition: " . ($a + $b);
        break;

    case "-":
        echo "Subtraction: " . ($a - $b);
        break;

    case "*":
        echo "Multiplication: " . ($a * $b);
        break;

    case "/":
        echo "Division: " . ($a / $b);
        break;

    default:
        echo "Invalid operation";
}
?>
```

Output

This will generate the below output

Addition: 15

Nested Switch (Switch Inside Another Switch)

A switch can be used inside another switch to make multi-level decisions. Now the below code show how you can use nested switch statements and see the outcome.

```
<?php
$continent = "Asia";
$country = "India";

switch ($continent) {
    case "Asia":
```

```
        switch ($country) {
            case "India":
                echo "You are in India!";
                break;
            case "Japan":
                echo "You are in Japan!";
                break;
            default:
                echo "Country not listed in Asia.";
        }
        break;

        case "Europe":
            switch ($country) {
                case "Germany":
                    echo "You are in Germany!";
                    break;
                case "France":
                    echo "You are in France!";
                    break;
                default:
                    echo "Country not listed in Europe.";
            }
            break;

        default:
            echo "Continent not recognized.";
    }
?>
```

Output

This will create the below output

You are in India!